

Complemento en R: red neuronal simple

Propagación hacia adelante, retropropagación y SGD

Red 1D con 3 unidades ocultas, activación $\tanh$ y salida lineal

Material auxiliar para conectar el código con las fórmulas de la Sesión 2

- ➊ Arquitectura usada en el script.
- ➋ Notación matemática y nombres en R.
- ➌ Fórmulas tomadas de la Sesión 2.
- ➍ Propagación hacia adelante.
- ➎ Retropropagación para el caso $H = 3$.
- ➏ Gradiente completo y actualización SGD.
- ➐ Correspondencia directa con el código.

Idea

El script en R implementa una red neuronal pequeña para aproximar una función de una variable:

$$x \in \mathbb{R} \quad \longmapsto \quad \hat{y} \in \mathbb{R}.$$

Propósito del documento

No volveremos a justificar toda la retropropagación. En su lugar, usaremos las fórmulas derivadas en la Sesión 2 y mostraremos cómo aparecen en el código.

Convención

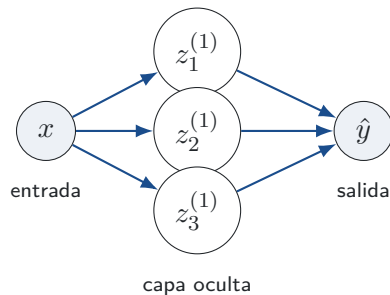
En las diapositivas fijamos $H = 3$ unidades ocultas para que la notación sea visible. En el código, este valor se puede cambiar mediante `hidden_size`.

Red del ejemplo

$$1 \longrightarrow 3 \longrightarrow 1$$

entrada $\longrightarrow$ capa oculta $\longrightarrow$ salida

- ▶ Entrada escalar: x .
- ▶ Tres unidades ocultas: $H = 3$.
- ▶ Activación oculta: $\tanh$.
- ▶ Salida lineal: $\hat{y} = a^{(2)}$.



Código R	Notación	Significado
W1	$W^{(1)}$	pesos de entrada a capa oculta
b1	$b^{(1)}$	sesgos de capa oculta
W2	$W^{(2)}$	pesos de capa oculta a salida
b2	$b^{(2)}$	sesgo de salida
a1	$a^{(1)}$	preactivación de capa oculta
z1	$z^{(1)}$	activación oculta después de tanh
a2	$a^{(2)}$	preactivación de salida
y_hat	$\hat{y}$	predicción final de la red

Convención

En este complemento usamos los sesgos **por separado**:

$$a^{(1)} = W^{(1)}x + b^{(1)}, \quad a^{(2)} = W^{(2)}z^{(1)} + b^{(2)}.$$

En la Sesión 2 también vimos la notación con sesgos absorbidos; ambas son equivalentes si se agrega una entrada fija igual a 1.

Para $H = 3$, las matrices y vectores del modelo son:

$$W^{(1)} = \begin{bmatrix} w_{11}^{(1)} \\ w_{21}^{(1)} \\ w_{31}^{(1)} \end{bmatrix}, \quad b^{(1)} = \begin{bmatrix} b_1^{(1)} \\ b_2^{(1)} \\ b_3^{(1)} \end{bmatrix}$$

$$W^{(2)} = \begin{bmatrix} w_{11}^{(2)} & w_{12}^{(2)} & w_{13}^{(2)} \end{bmatrix}, \quad b^{(2)} = \begin{bmatrix} b_1^{(2)} \end{bmatrix}.$$

Dimensiones

$$W^{(1)} \in \mathbb{R}^{3 \times 1}, \quad b^{(1)} \in \mathbb{R}^{3 \times 1}, \quad W^{(2)} \in \mathbb{R}^{1 \times 3}, \quad b^{(2)} \in \mathbb{R}^{1 \times 1}.$$

Sesgos

Como los sesgos no están absorbidos, sus gradientes se calculan aparte: $db^{(1)} = \partial E_n / \partial b^{(1)}$ y $db^{(2)} = \partial E_n / \partial b^{(2)}$.

Fórmulas de la Sesión 2 que usará el código

Para una conexión que entra a una unidad j :

$$\frac{\partial E_n}{\partial w_{ji}} = \delta_j z_i.$$

Para una red por capas con activación oculta $\tanh$:

$$\delta^{(\ell)} = \left(1 - (z^{(\ell)})^{\odot 2}\right) \odot \left((W^{(\ell+1)})^T \delta^{(\ell+1)}\right).$$

Salida lineal con pérdida cuadrática

$$E_n = \frac{1}{2}(\hat{y} - y)^2, \quad \delta^{(2)} = \frac{\partial E_n}{\partial a^{(2)}} = \hat{y} - y.$$

Sesgos por separado

En este complemento no absorbemos los sesgos en una entrada fija 1. Por eso escribimos

$$a_j = \sum_i w_{ji} z_i + b_j.$$

Entonces, para cada sesgo:

$$\frac{\partial E_n}{\partial b_j} = \frac{\partial E_n}{\partial a_j} \frac{\partial a_j}{\partial b_j} = \delta_j \cdot 1 = \delta_j.$$

Traducción al código

Los objetos db1 y db2 guardan esos gradientes de sesgo:

$$db^{(1)} = \frac{\partial E_n}{\partial b^{(1)}}, \quad db^{(2)} = \frac{\partial E_n}{\partial b^{(2)}}.$$

Forma vectorial usada por el código

$$a^{(1)} = W^{(1)}x + b^{(1)}$$

$$z^{(1)} = \tanh(a^{(1)})$$

$$a^{(2)} = W^{(2)}z^{(1)} + b^{(2)}$$

$$\hat{y} = a^{(2)}.$$

Código correspondiente

```
a1 <- W1%% x + b1,  z1 <- tanh(a1),  a2 <- W2%% z1 + b2.
```

Con $H = 3$, la ecuación matricial $a^{(1)} = W^{(1)}x + b^{(1)}$ equivale a:

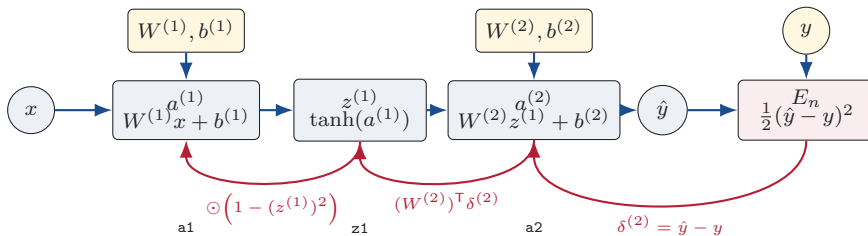
$$a_1^{(1)} = w_{11}^{(1)}x + b_1^{(1)}, \quad a_2^{(1)} = w_{21}^{(1)}x + b_2^{(1)}, \quad a_3^{(1)} = w_{31}^{(1)}x + b_3^{(1)}.$$

Después,

$$z_j^{(1)} = \tanh(a_j^{(1)}), \quad j = 1, 2, 3.$$

La salida es

$$\hat{y} = w_{11}^{(2)}z_1^{(1)} + w_{12}^{(2)}z_2^{(1)} + w_{13}^{(2)}z_3^{(1)} + b_1^{(2)}.$$



Azul: cálculo hacia adelante. Rojo: sensibilidades usadas para calcular los gradientes.

Pérdida para un dato

Para el dato de entrenamiento (x_n, y_n) :

$$E_n = \frac{1}{2}(\hat{y}_n - y_n)^2.$$

Derivada inicial

Como la salida es lineal, $\hat{y} = a^{(2)}$. Por tanto,

$$\delta^{(2)} = \frac{\partial E_n}{\partial a^{(2)}} = \frac{\partial E_n}{\partial \hat{y}} = \hat{y} - y.$$

Nota

El ECM que se grafica en el script se usa para monitorear el entrenamiento completo. La retropropagación de cada paso usa la pérdida de un solo dato E_n .

Gradientes de la segunda capa

De

$$a^{(2)} = W^{(2)} z^{(1)} + b^{(2)},$$

se obtiene

$$dW^{(2)} = \nabla_{W^{(2)}} E_n = \frac{\partial E_n}{\partial W^{(2)}} = \delta^{(2)} (z^{(1)})^T, \quad db^{(2)} = \nabla_{b^{(2)}} E_n = \delta^{(2)}.$$

Con $H = 3$

$$dW^{(2)} = \begin{bmatrix} \delta^{(2)} z_1^{(1)} & \delta^{(2)} z_2^{(1)} & \delta^{(2)} z_3^{(1)} \end{bmatrix}.$$

Delta de la capa oculta

Primero se regresa la sensibilidad desde la salida:

$$(W^{(2)})^T \delta^{(2)}.$$

Después se multiplica por la derivada local de $\tanh$:

$$\delta^{(1)} = \left((W^{(2)})^T \delta^{(2)} \right) \odot \left(1 - (z^{(1)})^2 \right).$$

Operador $\odot$

$\odot$ significa producto elemento a elemento. Si $u, v \in \mathbb{R}^3$, entonces

$$u \odot v = \begin{bmatrix} u_1 v_1 \\ u_2 v_2 \\ u_3 v_3 \end{bmatrix}.$$

Para cada unidad oculta $j = 1, 2, 3$:

$$\delta_j^{(1)} = \left(1 - (z_j^{(1)})^2\right) w_{1j}^{(2)} \delta^{(2)}.$$

Explícitamente:

$$\delta_1^{(1)} = (1 - (z_1^{(1)})^2) w_{11}^{(2)} \delta^{(2)},$$

$$\delta_2^{(1)} = (1 - (z_2^{(1)})^2) w_{12}^{(2)} \delta^{(2)},$$

$$\delta_3^{(1)} = (1 - (z_3^{(1)})^2) w_{13}^{(2)} \delta^{(2)}.$$

Correspondencia

Esto corresponde a `delta1 <- (t(W2) %*% delta2) * (1 - z1^2)`.

Gradientes de la primera capa

De

$$a^{(1)} = W^{(1)}x + b^{(1)},$$

se obtiene

$$dW^{(1)} = \nabla_{W^{(1)}} E_n = \frac{\partial E_n}{\partial W^{(1)}} = \delta^{(1)} x^T, \quad db^{(1)} = \nabla_{b^{(1)}} E_n = \delta^{(1)}.$$

Como en este ejemplo x es escalar:

$$dW^{(1)} = \begin{bmatrix} \delta_1^{(1)} x \\ \delta_2^{(1)} x \\ \delta_3^{(1)} x \end{bmatrix}, \quad db^{(1)} = \begin{bmatrix} \delta_1^{(1)} \\ \delta_2^{(1)} \\ \delta_3^{(1)} \end{bmatrix}.$$

Para el dato n

El gradiente completo que calcula `retropropagacion()` es:

$$\nabla_w E_n = \begin{bmatrix} \text{vec}(dW^{(1)}) \\ db^{(1)} \\ \text{vec}(dW^{(2)}) \\ db^{(2)} \end{bmatrix} = \begin{bmatrix} \text{vec}(\delta^{(1)} x^T) \\ \delta^{(1)} \\ \text{vec}(\delta^{(2)} (z^{(1)})^T) \\ \delta^{(2)} \end{bmatrix}.$$

En el código

El gradiente no se apila en un solo vector; se guarda como lista:

`dW1, db1, dW2, db2.`

```
propagacion_adelante(x, params)
```

- 1 Calcular $a1 \leftarrow W1 \%* \% x + b1$.
- 2 Calcular $z1 \leftarrow \tanh(a1)$.
- 3 Calcular $a2 \leftarrow W2 \%* \% z1 + b2$.
- 4 Fijar $y_hat \leftarrow a2$.
- 5 Devolver $x, a1, z1, a2, y_hat$ en una lista llamada cache.

Advertencia

La cache guarda las cantidades intermedias que se reutilizan en retropropagación.

```
retropropagacion(cache, y_real, params)
```

- ① Calcular $\delta_2 \leftarrow \hat{y} - y_{\text{real}}$.
- ② Calcular $dW_2 \leftarrow \delta_2 \otimes t'(z_1)$.
- ③ Calcular $db_2 \leftarrow \delta_2$.
- ④ Calcular $\delta_1 \leftarrow (t'(W_2) \otimes \delta_2) * (1 - z_1^2)$.
- ⑤ Calcular $dW_1 \leftarrow \delta_1 \otimes t'(x)$.
- ⑥ Calcular $db_1 \leftarrow \delta_1$.

Idea

Cada línea es una traducción directa de las fórmulas de la Sesión 2.

Actualización de parámetros

Con tasa de aprendizaje η :

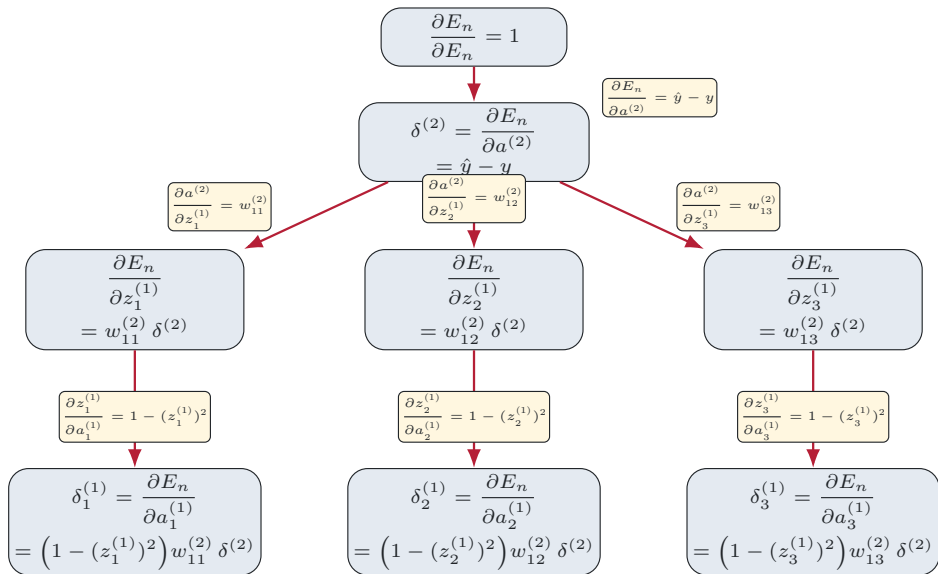
$$W^{(1)} \leftarrow W^{(1)} - \eta dW^{(1)}, \quad b^{(1)} \leftarrow b^{(1)} - \eta db^{(1)},$$

$$W^{(2)} \leftarrow W^{(2)} - \eta dW^{(2)}, \quad b^{(2)} \leftarrow b^{(2)} - \eta db^{(2)}.$$

Bucle de entrenamiento

- 1 Elegir un dato aleatorio (x_i, y_i) .
- 2 Hacer forward.
- 3 Hacer backprop.
- 4 Actualizar parámetros con SGD.
- 5 Calcular ECM en todo el conjunto para monitorear.

Apéndice: reverse mode sobre el grafo



- ▶ Se inicia con la derivada trivial $\partial E_n / \partial E_n = 1$.
- ▶ La primera sensibilidad relevante es

$$\delta^{(2)} = \frac{\partial E_n}{\partial a^{(2)}} = \hat{y} - y.$$

- ▶ Luego esa sensibilidad se reparte hacia cada activación oculta $z_j^{(1)}$ mediante los pesos de salida $w_{1j}^{(2)}$.
- ▶ Finalmente, al multiplicar por la derivada local de $\tanh$,

$$\frac{\partial z_j^{(1)}}{\partial a_j^{(1)}} = 1 - (z_j^{(1)})^2,$$

obtenemos las sensibilidades ocultas $\delta_j^{(1)}$.

- ▶ Esta es exactamente la lógica de *reverse-mode differentiation*: **derivada acumulada** $\times$ **derivada local**.